

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1: Constraint-Based Problem Solving

**COURSE NAME: COMPUTER VISION**

**COURSE CODE: ITA05**

### **COURSE OUTCOME COVERED**

**CO5:** *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 40%

**Constraint-Based Problem Solving:** Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

### **2. Automated Video Surveillance with Alert System**

A security firm needs a **video surveillance system** that:

- A. Detects motion and unusual activities
- B. Generates alerts in real time
- C. Works with standard CCTV video feeds
- D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

### **3. OCR System for Low-Quality Documents**

A digitization company needs an **OCR system** that:

- A. Extracts text from low-resolution, noisy scanned images
- B. Handles varying fonts and illumination
- C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

## 9. Document Scanner Application

A mobile app must:

- A. Capture document images
- B. Enhance readability
- C. Output clean scanned images

Design the OpenCV pipeline and justify preprocessing steps used for improving document quality.

## 10. Vehicle Detection in Parking System

A parking system must:

- A. Detect cars entering/exiting
- B. Track movement
- C. Update available slots

Design the pipeline and justify how your approach handles multiple vehicles and avoids counting errors.

## 15. Real-Time Edge Detection System

A system must:

- A. Capture video
- B. Detect edges in real time
- C. Display results efficiently

Design the pipeline and justify optimization strategies.

### **Code of Conduct:**

*I K.ASWINI(192312397) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**

# **RUBRICS FOR CALCULATION:**

| <b>Criteria</b>                | <b>Weight age</b> | <b>Level 4<br/>(Exemplary)</b>                                                        | <b>Level 3<br/>(Proficient)</b>                              | <b>Level 2<br/>(Developing )</b>                     | <b>Level 1<br/>(Beginning)</b>                  |
|--------------------------------|-------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------|------------------------------------------------------|-------------------------------------------------|
| <b>Problem Understanding</b>   | <b>25%</b>        | <b>Clearly defines the problem and identifies all relevant constraints accurately</b> | <b>Identifies most constraints with minor gaps</b>           | <b>Partial understanding; misses key constraints</b> | <b>Fails to identify constraints correctly</b>  |
| <b>Constraint Analysis</b>     | <b>25%</b>        | <b>Analyzes constraints effectively and prioritizes them logically</b>                | <b>Provides reasonable analysis with some prioritization</b> | <b>Limited analysis; weak prioritization</b>         | <b>No meaningful analysis or prioritization</b> |
| <b>Solution Development</b>    | <b>25%</b>        | <b>Develops optimal and feasible solutions satisfying all constraints</b>             | <b>Develops workable solutions with minor limitations</b>    | <b>Solutions partially satisfy constraints</b>       | <b>Solutions do not meet constraints</b>        |
| <b>Application of Concepts</b> | <b>15%</b>        | <b>Effectively applies appropriate concepts, methods, or tools</b>                    | <b>Applies concepts with minor errors</b>                    | <b>Limited application; requires guidance</b>        | <b>Unable to apply relevant concepts</b>        |
| <b>Justification</b>           | <b>10%</b>        | <b>Provides strong justification for decisions with logical</b>                       | <b>Provides justification with some clarity</b>              | <b>Weak or unclear justification</b>                 | <b>No justification provided</b>                |

|  |  |                  |  |  |  |
|--|--|------------------|--|--|--|
|  |  | <b>reasoning</b> |  |  |  |
|--|--|------------------|--|--|--|

## **ANSWERS**

### **2. Automated Video Surveillance with Alert System**

#### ◇ Pipeline Design

##### **1. Video Input**

1. Capture CCTV feed using `cv2.VideoCapture()`

##### **2. Preprocessing**

1. Convert to grayscale
2. Apply Gaussian blur → reduces noise

##### **3. Background Modeling**

1. Use `cv2.createBackgroundSubtractorMOG2()`
2. Separates foreground (moving objects)

##### **4. Motion Detection**

1. Threshold + morphological operations (dilation/erosion)
2. Extract contours → identify moving objects

##### **5. Feature Analysis (Unusual Activity)**

1. Object size, speed, direction
2. Sudden motion spikes or abnormal patterns

##### **6. Tracking**

1. Centroid tracking / Kalman Filter
2. Assign IDs to objects

##### **7. Alert Generation**

1. Trigger alert if:
  1. Object enters restricted area
  2. Abnormal speed/behavior detected

##### **8. Display Output**

## 1. Bounding boxes + alert labels

```
import cv2

cap = cv2.VideoCapture(0) # use CCTV/video file path if needed

fgbg = cv2.createBackgroundSubtractorMOG2()

while True:

    ret, frame = cap.read()

    if not ret:

        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    blur = cv2.GaussianBlur(gray, (5,5), 0)

    fgmask = fgbg.apply(blur)

    _, thresh = cv2.threshold(fgmask, 200, 255, cv2.THRESH_BINARY)

    thresh = cv2.dilate(thresh, None, iterations=2)

    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    for cnt in contours:

        area = cv2.contourArea(cnt)

        if area > 1000: # filter noise

            x,y,w,h = cv2.boundingRect(cnt)

            cv2.rectangle(frame,(x,y),(x+w,y+h),(0,255,0),2)

            # ALERT CONDITION

            cv2.putText(frame, "ALERT!", (x,y-10),

                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,0,255),2)

    cv2.imshow("Surveillance", frame)

    if cv2.waitKey(30) & 0xFF == 27:

        break

cap.release()

cv2.destroyAllWindows() ◇ Justification
```

- **Noise Handling:** Gaussian blur + morphology removes small disturbances
  - **Dynamic Background:** MOG2 adapts to lighting changes (trees, shadows)
  - **False Positives Reduction:** Filter by area + motion consistency
  - **Real-Time:** Lightweight operations (no heavy deep learning)
- 

### 3. OCR System for Low-Quality Documents

#### ◇ Pipeline Design

##### 1. Input Image

- Low-resolution scanned image

##### 2. Enhancement

- Resize (interpolation)
- Histogram equalization → improve contrast
- Bilateral filter → denoise while preserving edges

##### 3. Binarization

- Adaptive thresholding (cv2.adaptiveThreshold)
- Works under uneven lighting

##### 4. Noise Removal

- Morphological opening (remove speckles)

##### 5. Segmentation

- Contour detection → isolate text regions
- Line/word segmentation

##### 6. Text Normalization

- Deskew using Hough Transform
- Resize characters to standard size

##### 7. OCR Engine

- Pass processed image to Tesseract

```
import cv2
import pytesseract

# Load image
img = cv2.imread('low_quality_doc.jpg')

# Resize
img = cv2.resize(img, None, fx=2, fy=2, interpolation=cv2.INTER_CUBIC)

# Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Noise removal
blur = cv2.bilateralFilter(gray, 9, 75, 75)

# Adaptive threshold
thresh = cv2.adaptiveThreshold(blur, 255,
                               cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY, 11, 2)

# OCR
text = pytesseract.image_to_string(thresh)

print("Extracted Text:\n", text)

cv2.imshow("Processed", thresh)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

---

#### ◇ Justification

- **Low Quality Handling:** resizing + contrast enhancement improves clarity
  - **Noise:** bilateral filtering keeps edges sharp
  - **Illumination Variation:** adaptive thresholding solves uneven lighting
  - **Batch Efficiency:** pipeline is lightweight → scalable processing
- 

## 9. Document Scanner Application

#### ◇ Pipeline Design

##### 1. Capture Image

- Mobile camera input

## 2. Preprocessing

- Convert to grayscale
- Gaussian blur

## 3. Edge Detection

- Canny edge detection

## 4. Contour Detection

- Find largest 4-point contour (document boundary)

## 5. Perspective Transform

- Warp image to top-down view

## 6. Enhancement

- Adaptive threshold → “scanned” effect
- Sharpening filter

## 7. Output

- Clean scanned document

```
import cv2
```

```
import numpy as np
```

```
def order_points(pts):
```

```
    rect = np.zeros((4,2), dtype="float32")
```

```
    s = pts.sum(axis=1)
```

```
    rect[0] = pts[np.argmin(s)]
```

```
    rect[2] = pts[np.argmax(s)]
```

```
    diff = np.diff(pts, axis=1)
```



```
rect[1] = pts[np.argmin(diff)]
```

```
rect[3] = pts[np.argmax(diff)]
```

```
return rect
```

```
def four_point_transform(image, pts):
```

```
    rect = order_points(pts)
```

```
    (tl, tr, br, bl) = rect
```

```
    widthA = np.linalg.norm(br - bl)
```

```
    widthB = np.linalg.norm(tr - tl)
```

```
    maxWidth = max(int(widthA), int(widthB))
```

```
    heightA = np.linalg.norm(tr - br)
```

```
    heightB = np.linalg.norm(tl - bl)
```

```
    maxHeight = max(int(heightA), int(heightB))
```

```
    dst = np.array([
```

```
        [0,0],
```

```
        [maxWidth-1,0],
```

```
        [maxWidth-1,maxHeight-1],
```

```
        [0,maxHeight-1]], dtype="float32")
```

```
M = cv2.getPerspectiveTransform(rect, dst)
```

```
return cv2.warpPerspective(image, M, (maxWidth, maxHeight))
```

```
img = cv2.imread("document.jpg")
```

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
blur = cv2.GaussianBlur(gray,(5,5),0)
```

```
edges = cv2.Canny(blur, 75, 200)
```

```
contours, _ = cv2.findContours(edges, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)
```

```
contours = sorted(contours, key=cv2.contourArea, reverse=True)
```

```
for cnt in contours:
```

```
    peri = cv2.arcLength(cnt, True)
```

```
    approx = cv2.approxPolyDP(cnt, 0.02 * peri, True)
```

```
    if len(approx) == 4:
```

```
        doc = approx
```

```
        break
```

```
warped = four_point_transform(img, doc.reshape(4,2))
```

```
# enhance
```

```
warped_gray = cv2.cvtColor(warped, cv2.COLOR_BGR2GRAY)
```

```
scan = cv2.adaptiveThreshold(warped_gray,255,
```

```
cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
```

```
cv2.THRESH_BINARY,11,2)
```

```
cv2.imshow("Scanned", scan)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

◇ Justification

- **Edge Detection:** isolates document boundaries
  - **Perspective Correction:** removes skew
  - **Thresholding:** improves readability like a real scanner
  - **Mobile Efficiency:** fast operations → real-time use
- 

## 10. Vehicle Detection in Parking System

◇ Pipeline Design

1. **Video Capture**
2. **Preprocessing**
  - Resize + grayscale
3. **Vehicle Detection**
  - Background subtraction OR Haar Cascade / YOLO (optional)
4. **Tracking**
  - Centroid tracking / SORT algorithm
5. **Entry/Exit Detection**
  - Define virtual line

- Count vehicles crossing line

## 6. Slot Management

Maintain counter:

- 
- 

$\text{available\_slots} = \text{total} - \text{current\_vehicles}$

## 7. Display

- Show vehicle IDs + slot count .

```
import cv2
```

```
cap = cv2.VideoCapture("traffic.mp4")
```

```
fgbg = cv2.createBackgroundSubtractorMOG2()
```

```
line_position = 300
```

```
vehicle_count = 0
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
    fgmask = fgbg.apply(gray)
```

```

_, thresh = cv2.threshold(fgmask, 200, 255, cv2.THRESH_BINARY)

contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

cv2.line(frame, (0,line_position), (frame.shape[1],line_position), (255,0,0),2)

for cnt in contours:

    if cv2.contourArea(cnt) > 1500:

        x,y,w,h = cv2.boundingRect(cnt)

        cx = x + w//2

        cy = y + h//2

        if cy > line_position-5 and cy < line_position+5:

            vehicle_count += 1

            cv2.rectangle(frame,(x,y),(x+w,y+h),(0,255,0),2)

cv2.putText(frame, f'Count: {vehicle_count}', (50,50),
            cv2.FONT_HERSHEY_SIMPLEX,1,(0,0,255),2)

cv2.imshow("Parking System", frame)

if cv2.waitKey(30) & 0xFF == 27:

    break

cap.release()

```

```
cv2.destroyAllWindows()
```

◇ Justification

- **Multiple Vehicles:** tracking assigns unique IDs
- **Avoid Double Counting:** line-crossing logic with direction
- **Accuracy:** filters based on size and movement
- **Scalability:** works with standard CCTV feeds

## 15. Real-Time Edge Detection System

◇ Pipeline Design

1. **Video Capture**
2. **Frame Preprocessing**
  - Convert to grayscale
  - Apply Gaussian blur
3. **Edge Detection**
  - Canny edge detector
4. **Display Output**
  - Show edge frames continuously

```
import cv2
```

```
cap = cv2.VideoCapture(0)
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
    blur = cv2.GaussianBlur(gray, (5,5), 0)
```

```
    edges = cv2.Canny(blur, 50, 150)
```

```
cv2.imshow("Edges", edges)
```

```
if cv2.waitKey(1) & 0xFF == 27:
```

```
    break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

#### ◇ Optimization Strategies

- Use **lower resolution frames** → faster processing
- Process **every nth frame** instead of all frames
- Use **efficient filters (Gaussian instead of heavy methods)**
- Use **hardware acceleration (GPU/OpenCL if available)**

#### ◇ Justification

- **Real-Time Performance:** lightweight operations ensure speed
- **Noise Reduction:** blur before edge detection avoids false edges
- **Efficiency:** Canny is optimized and widely used